

Real-Time Embedded system I/O

The system has multiple sensors generating interrupts every 1ms. CPU utilization must remain below 5% and fast interrupt handling is required.

Constraint Analysis -

- CPU utilization < 5%.
- Sensor interrupts every 1ms
- Vectored interrupts required
- Low I/O latency required.

Memory-mapped I/O is selected because sensor registers can be accessed directly using normal load/store instructions.

Sensor → Interrupt controller → CPU → Sensor Registers



Vector Table

Each sensor has a separate interrupt vector. When an interrupt occurs, the CPU directly executes the corresponding ISR.

If one ISR takes 50ms:

$$\text{CPU Utilization} = \frac{50}{1000} \times 100 = 5\%$$

CPU utilization is below 15%.

Vectorized interrupts provides fast response while memory-mapped I/O gives simple & low-latency register access.

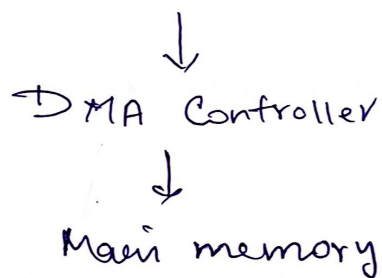
Q2. NVMe over PCIe with DMA.

The storage system requires throughput above 3GB/s and the CPU should not handle bulk data transfer.

Constraint Analysis

- Throughput > 3GB/s
- NVMe over PCIe
- Minimum CPU involvement
- DMA with interrupt notification

CPU → NVMe Driver → PCIe → NVMe SSD



Completion Interrupt.

The CPU starts the transfer, DMA moves the data directly between SSD and memory, and an interrupt informs the CPU after completion.

NVMe over PCIe provides high-speed storage, while DMA reduces CPU workload and supports high throughput.

fact

USB and Thunderbolt Communication

The System Contains 6 USB peripherals and 2 Thunderbolt devices. Communication must be fair and interrupt starvation must be avoided.

um

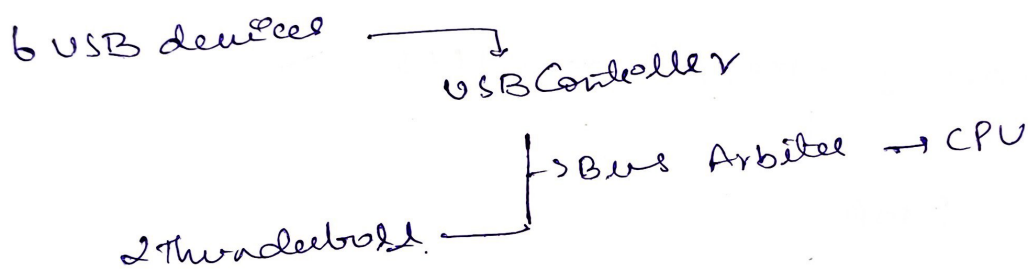
re

er

e

Constraint Analysis

- 6 USB devices
- 2 Thunderbolt devices
- hardware & software interrupts
- No interrupt starvation
- Fair bus access.



A round robin bus arbitration method gives each requesting device a turn.

- Hardware interrupts → device events
- Software interrupts → background processes
- Round-robin → Prevents starvation.

Round-robin arbitration ensures fair bus access, while hardware & software interrupts handle different types of requests efficiently.

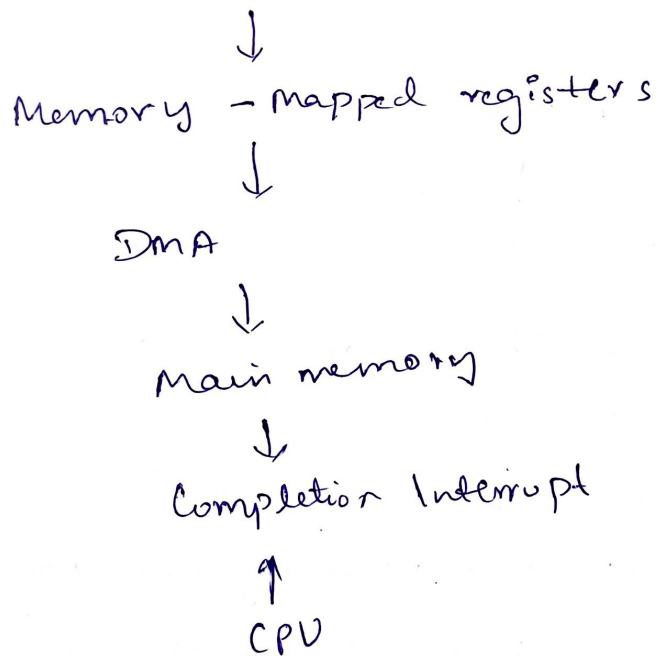
Q6. Data Acquisition System

The System continuously generates data at 500MB/s. CPU intervention must be minimal.

Constraint Analysis

- Continuous 500MB/s Stream
- Minimum CPU involvement
- DMA vs interrupt-driven I/O Comparison
- Memory-mapped device registers.

Data Acquisition Device



DMA

low CPU usage

Suitable for bulk data

Transfers block directly

Interrupt-Driven I/O

high CPU usage

less suitable for
500 MB/s

CPU handles interrupts

DMA is selected because it transfers large amount of data to memory & reduces CPU overhead.

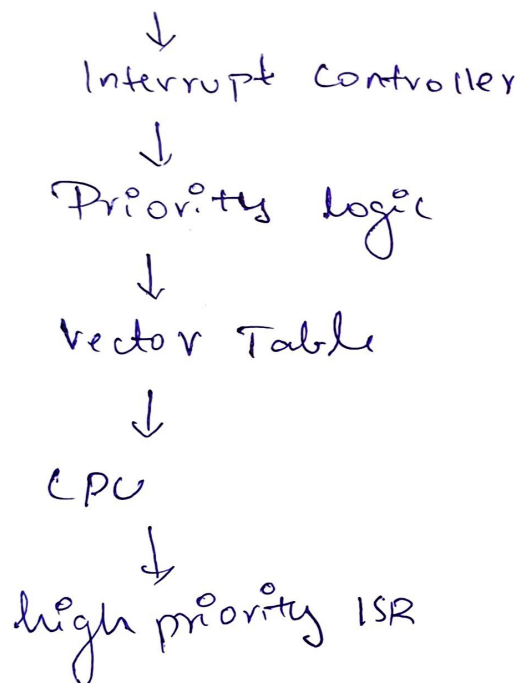
Nested Interrupt Handling:

The system must support nested interrupts and handle high-priority devices within 100ns.

Constraint Analysis

- Nested interrupts
- Latency < 100ns
- Vectored interrupts for high priority
- Software interrupts for background tasks.

Devices



If a low-priority ISR is running and a high-priority interrupt arrives, the CPU temporarily pauses the low-priority ISR and handles the high-priority ISR.

Eg Latency: $2 + 1 + 2 + 2 = 7\mu s$

$7\mu s < 100ns$

Nested and vectored provide fast handling of critical events, while software interrupts are suitable for low-priority background operations.